

Ejercicio 4

Descargar este enunciado [pdf](#) o [html](#)



Muy Importante

Deberemos usar typesafe routes para la navegación. Esto es, las rutas y sus argumentos, **en ningún caso serán cadenas de texto**, sino que serán objetos que definiremos en el paquete `ui.navigation`.

Partiendo del [Ejercicio 3](#), se pide realizar las siguientes tareas:

Implementación de la **navegación** y como la primera pantalla `GaleriaCochesScreen` poder navegar de esta a `FichaCocheScreen` al pulsar la acción de ver la ficha de un coche, y la gestión de volver atrás. Además, desde la pantalla de la ficha de un coche, se podrá navegar al diálogo `DescuentoDialog` desde el botón de Ofertas.

Para ello, crearemos el paquete `ui.navigation` y dentro del mismo los siguientes archivos:

- `VehiculosNavHost.kt` que contendrá el composible a emitir desde el MainActivity con el `NavHost` que define las rutas de navegación y crea las instancias de los ViewModels usando `hiltViewModel`. Además, será el encargado definir los callbacks que usando el `navController` me permitan navegar entre pantallas. En este caso, ir de la galería de coches a la ficha de un coche o volver atrás con `navigateUp()`.



Consejo

Para facilitar la navegación y gestionarla desde los ViewModels a través del bundle de eventos, lo más sencillo es definir los callbacks en el mismo `VehiculosNavHost` y guardarlos en los ViewModels para ser llamados desde dichos eventos. De esta forma, abstraemos a los ViewModels de la navegación usada y evitamos acoplarlos a una navegación concreta.

```
// Guadamos los callbacks con la navegación en los VM
// para ser llamados desde los eventos. Abstrayendo así el VM de la navegación usada.
vmGc.onNavegarFichaCoche = { id -> navController.navigate(FichaCocheRoute(id)) }
vmFc.onVolverAtras = {
    navController.popBackStack()
    // Esta llamada se evitaría usando StateFlows
    // pues proporciona baja cohesión y evitaría
    // llamadas a la fuente de datos de forma innecesaria.
    vmGc.onActualizarCoches()
}
vmFc.onNavegarDialogoDescuento = { navController.navigate(DialogoDescuentoRoute) }
```

- **GaleriaCochesRoute.kt** que será el destino inicial de la navegación, definirá su ruta y emitirá **GaleriaCochesScreen** .
- **FichaCocheRoute.kt** que será el destino de la navegación al editar la ficha de un coche de la galería, definirá su ruta que recibe como parámetro el id del coche a editar y emitirá **FichaCocheScreen** .
- **DialogoOfertaRoute.kt** que será el destino de la navegación al pulsar el botón de ofertas en la ficha de un coche, definirá su ruta y emitirá **DescuentoDialog** . Ten en cuenta que para definir que el destino se abra como un diálogo tenemos que usar **dialog<DialogoDescuentoRoute> {...** en lugar de **composable<FichaCocheRoute> {...** .